

Spec-Driven Delivery

Every feature is written down before it is built, verified against what was written, and blocked from shipping until it proves itself. The method is enforced by machinery, not by discipline.

Stack .NET, React, SQLite, containerised

In use on 46 active projects

Revised 12 September 2026

Why a method at all

AI-assisted development is fast and confidently wrong. It writes code that compiles, passes the tests it wrote for itself, and quietly implements nine of the twelve behaviours the requirement described. The failure is never loud — it looks exactly like finished work.

Three defects account for most of it: the requirement was never written down precisely enough to check against; the tests covered the happy path of a subset of the functions; and nobody could tell the difference between a check that passed and a check that never ran.

Everything below exists to close one of those three. The design principle throughout is that **a rule nobody can enforce is a rule nobody follows** — so each requirement is paired with something deterministic that refuses to proceed without it.

What happens to a requirement

Twelve phases. Each produces an artifact the next one reads. The chain runs to completion as a single unit of work — there are no approval checkpoints between phases, because the work was already approved when it was commissioned.

01 **specify**

The requirement becomes a written specification: scope, actors, data, the states a user can observe. Nothing is implemented from a conversation.

-
- 02 **interview**
- 15–25 recorded questions against the spec: scope boundaries, validation rules, authorization, concurrency, error semantics, edge cases, non-goals, reversibility. This is the anti-drift pass — it is where an assumption that would have become a defect gets named instead.
- *hard floor: 15 answers*
-
- 03 **clarify**
- Residual ambiguity the interview did not reach. Anything genuinely undecidable is escalated to you rather than guessed.
-
- 04 **elicit**
- The spec is restated as a formal model — entities, transitions, invariants. This is the baseline that later makes *drift* detectable: a difference between what was specified and what was built.
- *skipped on non-behavioural work*
-
- 05 **plan**
- Architecture and approach, derived from the clarified spec rather than from habit.
-
- 06 **tasks**
- The functional inventory: every behaviour the spec describes, itemised. This list is what test coverage is later measured against — which is how "nine of twelve" becomes visible.
-
- 07 **analyze**
- Consistency check across spec, plan and tasks. Contradictions are resolved before any code exists.
-
- 08 **implement**
- The code, written against the task list and the patterns already in the codebase.

-
- 09 **converge**
- The codebase is re-assessed against the spec and any unbuilt work is appended back to the task list. It loops until it appends nothing. This is the mechanical catch for partial implementation — it does not depend on anyone noticing.
-
- 10 **simplify**
- A quality-only pass on the changed code — reuse, duplication, altitude. Run before the tests are written, so the tests are written against the code that will actually ship.
-
- 11 **test**
- Unit, integration, end-to-end, property-based, visual regression, and a destructive suite sized per interactive function. Detailed below.
-
- 12 **verify**
- Formal verification with TLA+ on state machines and concurrency: race conditions, unreachable states, missing invariants — the class of defect testing is structurally bad at finding, because it requires guessing the interleaving.
- *state machines & concurrency*
-

Four tracks, chosen by risk

Running the full chain on a dependency bump is waste, and waste is how a process gets abandoned. Each specification is triaged at the point it is written.

Track	When	What runs
hardened	Authentication, payments, personal data, file upload, a new external surface, or a new entity	The full chain <i>plus</i> a threat model over each new trust boundary, an expanded destructive and load suite, a blocking mutation-score gate, and two independent adversarial reviews
full	New behaviour, new state machine, new concurrency	All 12 phases

Track	When	What runs
light	Single-actor UI work with no concurrency — a form, a filter, a linear flow	All but formal verification
spec-only	Refactor, documentation, configuration, dependency bump	Spec and clarify, then implement. No formal model.

Hardened is additive, not alternative. A payments feature runs everything a full-track feature runs and then four more checks on top. Over-applying it costs review time; under-applying it costs a breach, so the tie is broken toward hardening.

How the method is held in place

The method is not documentation that people are asked to follow. It is 59 automated hooks across 8 lifecycle events, in three tiers distinguished by what they can actually do.

Blocks

Six deterministic refusals

These do not warn — they deny the operation. Editing application source is refused outright until the specification exists, until its phases are complete, and until the interview records its answers. Three further guards protect shared machinery and refuse writes to sensitive files.

They fail *closed*: a guard that cannot establish what it is guarding refuses rather than waves work through.

Reminds

Context, not interruption

Coverage gaps, scenario drift, missing formal models and stale verification are surfaced to the agent as it works, once per subject per session. They inform the work without stopping it, and without filling the operator's screen.

Proves **The harness tests itself**

36 test suites and 5 validators — roughly 580 assertions — run against the enforcement machinery itself, so a gate that has silently stopped working is caught. This matters more than it sounds: a guard that permits everything and a guard with nothing to stop produce identical output, and only a test can tell them apart.

What "tested" is required to mean

Test count is not evidence. A suite can be large, green, and prove nothing. Six layers are required, each answering a question the others cannot.

Layer	What it proves	Sizing
Unit	The logic is right in isolation	per function
Integration	The seams hold — where AI-written code most often fails, because the parts pass and the joins do not	per boundary
End-to-end	The feature works in a real browser against a real stack	one per behaviour
Destructive	It fails correctly — invalid input, wrong order, skipped steps, boundaries, races	3 → 30+
Property-based	It holds across a generated input domain, not three chosen examples	wide-input logic
Visual regression	The interface did not change where it was not meant to	per key state

Destructive tests are sized, not counted

The suite for an interactive element is derived from its input domain, using equivalence partitioning and boundary analysis. A toggle earns about three tests; a multi-step authenticated flow earns twenty to thirty. A flat quota per feature is how a login form and a checkbox end up equally tested.

The mutation score is the real gate

Faults are injected into the changed code and the suite is asked to catch them. The kill rate — target ~80% on critical modules — is what proves the tests actually bite. On hardened work it blocks completion outright; elsewhere it runs on demand, because it is expensive and a check that costs an afternoon is a check that gets skipped.

Every interactive element must prove four states at runtime — success, a specific and visible error, empty, and loading. A failure that is silent counts as a defect, not as an edge case. "It works" means observed working, not compiled.

One list, one item at a time

Every project carries an ordered register of the specifications planned for it. It is the single source of truth for what gets built, in what order, and how far the work has come.

```
- [x] 001 — user-auth — full track [hardened] — sign-in, sessions, logout
- [x] 002 — profile-page — light track — view and edit own profile
- [/] 003 — search — full track — free-text search over the catalogue
- [ ] 004 — admin-dashboard — full track — operator overview
- [!] 005 — billing — full track [hardened] — blocked: VAT rules unconfirmed
- [ ] H1 — integration-hardening — checkpoint — full regression + security sweep
```

- [] Not started
- [/] In progress — only ever one at a time
- [x] Done, tested, committed, pushed
- [!] Held — and the row says why, so the reason survives the week

One stop per specification

A specification runs end to end without interruption, then stops with a status summary: which track, what was tested, what was found, what is next. That stop is the natural reporting point — frequent enough to steer, rare enough to make progress.

The backlog has to converge

Good gates find problems faster than a team can close them, and a backlog that grows faster than it drains never finishes regardless of effort. The ratio of items added to items closed is measured; past a threshold the project stops and the list is renegotiated rather than quietly extended.

A checkpoint every fifth item

Five individually correct features can still interact badly. After every fifth, a full-system pass runs: complete regression, a security sweep across the whole surface, reconciliation of the scenario map, and a mutation spot-check on what changed most.

Findings are batched, not acted on

A finding is recorded and decided at the next review, in a batch, where a dozen separate observations often turn out to be one thing. A pipeline that acts on every finding the moment it appears is a pipeline planning its own work.

What has to be true before we say it works

- Every scenario in the specification observed working at runtime — real behaviour, not a stub — with all four observable states proven.
- Unit and integration tests green. Property-based tests present for wide-input logic.
- End-to-end tests green, with one test per implemented behaviour and a destructive suite sized to each interactive element.
- Visual-regression baselines recorded for the key states.
- Mutation score on the changed critical modules at target.
- Formal verification run on state machines and concurrency.
- Clean build, full suite green, and the application running both locally and in its container — because the container is the artifact that ships.
- Visually confirmed in a browser.

If any of these could not be run, we say so explicitly rather than reporting a pass. **An unrun check is never reported as a passed one** — that rule is enforced in the tooling's own exit codes, not only in the prose.

Shipping and infrastructure

One image, one cluster

Applications build to a single container — .NET backend with the React frontend compiled into it — and deploy to a Docker Swarm cluster. Deployment is deliberate and manually triggered, never automatic on a commit.

Validated locally before it leaves

The full suite, the security scan and the container run happen on the developer's machine before anything is deployed. Continuous-integration minutes are spent on the deployment itself, not on re-buying information already in hand — a deliberate cost discipline after a previous project exhausted a month's budget in four days.

Built for interruption

The worker fleet runs on pre-emptible capacity that can be reclaimed at short notice. Stateful services are designed around that rather than assuming it away, with durable state on shared storage and documented constraints for the database layer.

What this method costs

It is front-loaded. A feature that a less formal process would start coding within the hour spends that hour being specified, interviewed and planned. That time is recovered on anything non-trivial and is not recovered on a genuinely tiny change — which is why trivial work is allowed to bypass the chain, provided the classification is stated on the record.

It is deliberately serial. One specification is worked at a time so that what is in flight is always knowable. This trades peak throughput for the ability to answer "what is the state of the project" precisely, at any moment.

The expensive proofs run on a cadence, not on every commit. Mutation testing, full-system regression and the dependency and secret scans are scheduled against what actually invalidates them and presented when due. They are tracked so they cannot be forgotten, and they are not pretended to be continuous.

And the machinery is itself under test, because we have found it wrong. Gates have been discovered inert — present in configuration, correct-looking, and doing nothing. That is the failure mode this whole approach is built against, and finding one in our own tooling is the reason the harness carries a test suite of its own.

What the system is made of

12

pipeline phases, each producing an artifact
the next one reads

59

automated hooks across 8 lifecycle events

6

deterministic blocks that refuse rather than
warn

21

enforced engineering rules, loaded every
session

41

test suites and validators over the machinery
itself

46

projects running this pipeline today

The method ships as a versioned configuration that every project synchronises from. An improvement found on one project reaches all of them — and so does a defect, which is why the shared machinery is the most heavily tested part of the system.

Prepared as a reference for clients asking how our engineering work is organised. Figures are counted from the working system on 12 September 2026, not estimated. Infrastructure specifics, credentials and client-identifying details are deliberately excluded.